



Phase 1: Data Understanding & Cleaning

CONTEXT — What this phase is and why it matters

Data cleaning is the most important step in every ML project. Dirty data leads to incorrect models (“Garbage In, Garbage Out”).

Importance in medicine:

- Medical data are typically full of missing values
- Outliers may be biological, not errors
- Data quality directly affects clinical decisions

This phase includes:

1. Fully understand the data structure
 2. Identify and handle missing values
 3. Detect and handle outliers
 4. Check for duplicates and inconsistencies
 5. Initial feature engineering
 6. Encoding and scaling
-

INPUTS — Required input files

Required files:

[path]

Optional files:

- `Phase0_Setup_Report.md` — Report from the previous phase
-

TASKS — What needs to be done



TASK 1: Read and fully understand the dataset

```

import pandas as pd
import numpy as np

# Load data
df = pd.read_csv("[path]")

print("=" * 80)
print("📊 COMPREHENSIVE DATA UNDERSTANDING")
print("=" * 80)

# Basic info
print(f"\n📐 Shape: {df.shape[0]} rows × {df.shape[1]} columns")
print(f"\n📋 Column Names:\n{list(df.columns)}")

# First few rows
print(f"\n🔍 First 5 Rows:")
display(df.head())

# Data types
print(f"\n📊 Data Types:")
print(df.dtypes)

# Memory usage
print(f"\n📦 Memory Usage: {df.memory_usage(deep=True).sum() / 1024**2:.2f} MB")

```

📋 TASK 2: Intelligent data-type detection

For each column, determine:

- **Numerical Continuous:** age, hormones, measurements
- **Numerical Discrete:** counts, tallies
- **Categorical Nominal:** gender, ethnicity, disease type
- **Categorical Ordinal:** grade, level, stage
- **Binary:** Yes/No, 0/1, Success/Failure
- **Text:** descriptions, names
- **Date/Time:** dates

```

def intelligent_dtype_detection(df):
    """Intelligent data-type detection."""
    dtype_info = {}

    for col in df.columns:
        unique_count = df[col].nunique()
        total_count = len(df[col].dropna())
        unique_ratio = unique_count / total_count if total_count > 0 else 0

        # Check for binary
        if unique_count == 2:
            dtype_info[col] = "Binary"
        # Check for categorical
        elif unique_count <= 10 or unique_ratio < 0.05:
            dtype_info[col] = "Categorical"
        # Check for text
        elif df[col].dtype == 'object' and df[col].str.len().mean() > 20:
            dtype_info[col] = "Text"
        # Check for date
        elif df[col].dtype == 'object':
            try:
                pd.to_datetime(df[col].dropna().iloc[0])
                dtype_info[col] = "DateTime"
            except Exception:
                dtype_info[col] = "Categorical"
        # Numerical
        else:
            if df[col].dtype in ['int64', 'int32']:
                dtype_info[col] = "Numerical_Discrete"
            else:
                dtype_info[col] = "Numerical_Continuous"

    return dtype_info

dtype_info = intelligent_dtype_detection(df)
print("\n📊 Intelligent Data Type Detection:")
for col, dtype in dtype_info.items():
    print(f"    • {col}: {dtype}")

```

TASK 3: Missing-values analysis

```

def comprehensive_missing_analysis(df):
    """Comprehensive missing-values analysis."""
    missing_df = pd.DataFrame({
        'Column': df.columns,
        'Missing_Count': df.isnull().sum().values,
        'Missing_Percent': (df.isnull().sum() / len(df) * 100).values,
        'Non_Missing': df.notnull().sum().values
    })

    missing_df = missing_df[missing_df['Missing_Count'] > 0].sort_values(
        'Missing_Percent', ascending=False
    )

    print("\n🔍 MISSING VALUES ANALYSIS")
    print("=" * 60)

    if len(missing_df) == 0:
        print("✅ No missing values found!")
    else:
        print(missing_df.to_string(index=False))

        # Severity classification
        critical = missing_df[missing_df['Missing_Percent'] > 50]
        high = missing_df[(missing_df['Missing_Percent'] > 20) & (missing_df['Missing_Percent'] <= 50)]
        moderate = missing_df[(missing_df['Missing_Percent'] > 5) & (missing_df['Missing_Percent'] <= 20)]
        low = missing_df[missing_df['Missing_Percent'] <= 5]

        print(f"\n⚠️ Severity Classification:")
        print(f"🔴 Critical (>50%): {list(critical['Column'])}")
        print(f"🟡 High (20-50%): {list(high['Column'])}")
        print(f"🟡 Moderate (5-20%): {list(moderate['Column'])}")
        print(f"🟢 Low (<5%): {list(low['Column'])}")

    return missing_df

missing_analysis = comprehensive_missing_analysis(df)

```

Missing-values handling strategies:

```

def handle_missing_values(df, strategy='intelligent'):
    """
    Missing-values handling strategies:

    1. Deletion:
        - Listwise: drop the entire row
        - Pairwise: drop only for the specific analysis
        - Column: drop columns with > 50% missing values

    2. Imputation:
        - Mean / Median: for numerical features
        - Mode: for categorical features
        - KNN imputation: based on similar samples
        - Multiple imputation: MICE
        - Regression imputation: predict missing values

    3. Advanced:
        - Indicator variable: create a missing-indicator column
        - Domain knowledge: based on medical domain rules
    """
    df_clean = df.copy()

    for col in df.columns:
        if df[col].isnull().sum() == 0:
            continue

        missing_pct = df[col].isnull().sum() / len(df) * 100

        # Critical missing (>50%) - consider dropping the column
        if missing_pct > 50:
            print(f" ⚠ Column '{col}' has {missing_pct:.1f}% missing – recommend dropping")
            continue

        # Choose imputer by dtype
        if df[col].dtype in ['float64', 'int64']:
            if df[col].skew() > 1 or df[col].skew() < -1:
                df_clean[col] = df[col].fillna(df[col].median())
            else:
                df_clean[col] = df[col].fillna(df[col].mean())
        else:
            df_clean[col] = df[col].fillna(df[col].mode().iloc[0])

    return df_clean

```

TASK 4: Outlier detection and handling

Outlier-handling strategies:

```
def detect_outliers(df, method='iqr'):
    """Outlier detection using multiple methods."""
    outliers_info = {}

    for col in df.select_dtypes(include=[np.number]).columns:
        if method == 'iqr':
            Q1 = df[col].quantile(0.25)
            Q3 = df[col].quantile(0.75)
            IQR = Q3 - Q1
            lower = Q1 - 1.5 * IQR
            upper = Q3 + 1.5 * IQR
            mask = (df[col] < lower) | (df[col] > upper)
        elif method == 'zscore':
            z = (df[col] - df[col].mean()) / df[col].std()
            mask = z.abs() > 3
        else:
            continue

        outliers_info[col] = {
            'count': int(mask.sum()),
            'percent': float(mask.sum() / len(df) * 100),
            'lower': float(df[col].min()),
            'upper': float(df[col].max())
        }

    return outliers_info
```

Decision rules for outliers:

- < 1%: usually safe to keep — may be biologically valid
- 1-5%: investigate; cap (winsorize) if appropriate
- > 5%: review per-feature; rare in clinical data
- **Always**: consult the clinical team before dropping

TASK 5: Duplicates and inconsistencies check

```
def check_duplicates(df):
    """Identify exact and near-duplicate rows."""
    exact = df.duplicated().sum()
    print(f"Exact duplicate rows: {exact}")

    # Check for near-duplicates by primary key (e.g. patient ID)
    if 'patient_id' in df.columns:
        near = df.duplicated(subset=['patient_id'], keep=False).sum()
        print(f"Rows sharing patient_id: {near}")

    return df.drop_duplicates()
```

Inconsistency checks:

- Range validation (age 0–120, BMI 10–60, etc.)
- Logical relationships (pregnancy + male = invalid)
- Unit consistency (mg/dL vs mmol/L)
- Categorical consistency (Male/M/male should be unified)

TASK 6: Data-type corrections

```
def correct_dtypes(df, dtype_info):
    """Apply correct dtypes based on detected types."""
    df_corrected = df.copy()

    for col, dtype in dtype_info.items():
        try:
            if dtype == 'Numerical_Continuous':
                df_corrected[col] = pd.to_numeric(df[col], errors='coerce')
            elif dtype == 'Numerical_Discrete':
                df_corrected[col] = pd.to_numeric(df[col], errors='coerce').astype('Int64')
            elif dtype == 'DateTime':
                df_corrected[col] = pd.to_datetime(df[col], errors='coerce')
            elif dtype == 'Categorical':
                df_corrected[col] = df[col].astype('category')
        except Exception as e:
            print(f" ⚠️ Could not convert {col}: {e}")

    return df_corrected
```

TASK 7: Initial feature engineering

```

def initial_feature_engineering(df):
    """Domain-driven initial feature engineering."""
    df_fe = df.copy()

    # Hormonal ratios (clinical relevance for NOA)
    if {'FSH', 'LH'}.issubset(df.columns):
        df_fe['FSH_LH_ratio'] = df_fe['FSH'] / df_fe['LH'].replace(0, np.nan)

    if {'Testosterone', 'LH'}.issubset(df.columns):
        df_fe['Test_LH_ratio'] = df_fe['Testosterone'] / df_fe['LH'].replace(0, np.nan)
    )

    # Age groups
    if 'Age' in df.columns:
        df_fe['Age_group'] = pd.cut(df_fe['Age'], bins=[0, 30, 40, 50, 100],
                                    labels=['<30', '30-40', '40-50', '50+'])

    # Binary indicators
    if 'FSH' in df.columns:
        df_fe['High_FSH'] = (df_fe['FSH'] > 7.6).astype(int)
    if 'Testosterone' in df.columns:
        df_fe['Low_Testosterone'] = (df_fe['Testosterone'] < 3.0).astype(int)

    # BMI
    if {'Body_Weight', 'Height'}.issubset(df.columns):
        df_fe['BMI'] = df_fe['Body_Weight'] / (df_fe['Height'] / 100) ** 2

    return df_fe

```

TASK 8: Encoding categorical variables

```

from sklearn.preprocessing import LabelEncoder, OneHotEncoder

def encode_categorical(df, dtype_info):
    """Encoding strategy:
    - Binary -> LabelEncoder
    - Categorical (low cardinality) -> One-Hot
    - Categorical (high cardinality) -> Target encoding
    """
    df_enc = df.copy()
    encoders = {}

    for col, dtype in dtype_info.items():
        if dtype == 'Binary':
            le = LabelEncoder()
            df_enc[col] = le.fit_transform(df_enc[col].astype(str))
            encoders[col] = le
        elif dtype == 'Categorical':
            n_unique = df_enc[col].nunique()
            if n_unique <= 10:
                df_enc = pd.get_dummies(df_enc, columns=[col], prefix=col)
            else:
                # Use target encoding or hashing for high cardinality
                pass

    return df_enc, encoders

```

TASK 9: Scaling numerical features

```

from sklearn.preprocessing import StandardScaler, RobustScaler, MinMaxScaler

def scale_features(df, num_cols, method='standard'):
    """Scaling strategies:
    - StandardScaler: when distribution is approximately normal
    - RobustScaler: when outliers are present
    - MinMaxScaler: when bounded range is required (e.g., neural networks)
    """
    if method == 'standard':
        scaler = StandardScaler()
    elif method == 'robust':
        scaler = RobustScaler()
    elif method == 'minmax':
        scaler = MinMaxScaler()
    else:
        raise ValueError(f"Unknown scaler: {method}")

    df_scaled = df.copy()
    df_scaled[num_cols] = scaler.fit_transform(df[num_cols])
    return df_scaled, scaler

```

TASK 10: Save the cleaned dataset and report

```
# Save outputs
df_clean.to_csv('1_Data/Cleaned/cleaned_dataset.csv', index=False)
df_encoded.to_csv('1_Data/Processed/encoded_dataset.csv', index=False)

# Save figures
# - missing_values_analysis.png/.tiff
# - outliers_boxplot.png/.tiff
# - data_types_summary.png/.tiff

# Generate Phase1_Report.md with:
# - Dataset overview
# - Data-quality summary
# - Cleaning decisions
# - Feature-engineering summary
# - Final encoded shape
```

INTELLIGENT DECISIONS

The agent makes data-driven decisions for:

- Imputation strategy per column (median vs mean vs mode vs drop)
 - Outlier policy (keep, cap, drop) based on domain heuristics
 - Encoding scheme (label, one-hot, target) based on cardinality
 - Scaling method (standard, robust) based on outlier presence and distribution
-

QUESTION MECHANISM — How to ask the user

When the agent encounters ambiguous decisions (e.g., a column with 35% missing values), it asks the user once with a clear recommendation rather than guessing silently.

OUTPUTS — Outputs of this phase

- 1_Data/Cleaned/cleaned_dataset.csv
 - 1_Data/Processed/encoded_dataset.csv
 - 3_Results/Phase1_DataCleaning/Phase1_Report.md
 - Figures: missing_values_analysis , outliers_boxplot , data_types_summary (PNG + TIFF)
-

OUTPUT FORMAT — Report format

Executive Summary

A concise summary of dataset shape, data-quality findings, and key cleaning decisions.

1. Dataset Overview

Rows, columns, memory usage, target distribution.

2. Data Types Identified

Tabular summary mapping columns to inferred types.

3. Missing Values

Per-column missing counts, percentages, and chosen imputation strategy.

4. Outliers

Detection method, counts, and treatment per column.

5. Feature Engineering

Newly created features, the rationale for each, and the resulting feature set.

6. Encoding & Scaling

Encoding scheme per categorical, scaler per numerical, and dataset shape after preprocessing.

7. Cleaned-Dataset Snapshot

First and last few rows of the cleaned dataset for sanity checks.